

Universidad Autónoma de Chihuahua

Facultad de ingeniería

GRUPO:

5HW1

Sistemas Operativos

Proyecto final Modulo de kernel

NOMBRE DE LOS ALUMNOS:

Nicolás Paúl Nevárez Loera

Samuel Eduardo García Gómez

MATRICULA:

367886

367651

Chihuahua, Chih. 26 de Noviembre del 2025

Reporte de Proyecto Final: Módulo de Kernel "FinalKernel"

1. Funcionamiento Detallado del Módulo

El proyecto consiste en el desarrollo e implementación de un módulo de kernel para Linux (Character Device Driver) denominado *finalkernel*. Este módulo gestiona un dispositivo virtual de caracteres que permite la comunicación bidireccional entre el espacio de usuario y el espacio de kernel.

Características Técnicas Principales:

1. Inicialización Robusta:

- Se utiliza la función `alloc_chrdev_region` para la asignación dinámica de números mayores (Major Numbers).
- Implementa la creación automática del nodo en `/dev` mediante `class_create` y `device_create`, eliminando la necesidad de usar `mknod` manualmente.
- Manejo de Errores: Se implementó una estructura de etiquetas goto (estilo cascada) en la función de inicio para asegurar una limpieza correcta de memoria en caso de fallos durante la carga.

2. Lógica de Control de Flujo (Stream):

- El dispositivo opera en dos modos distintos controlados por una bandera de estado interna:
 - Modo Normal (Default): Envía el mensaje predefinido una sola vez y cierra la transmisión (EOF).
 - Modo Flujo (Stream): Envía el mensaje de manera continua e infinita hasta que el usuario interrumpe el proceso.

3. Interacción de Entrada/Salida:

- Lectura (read): Transfiere datos del kernel al usuario usando `copy_to_user`.
- Escritura (write): Recibe comandos del usuario mediante `copy_from_user` para cambiar el estado del módulo:
 - Entrada '1': Activa el modo flujo.
 - Entrada '0': Desactiva el modo flujo.

2. Guía de Uso y Comandos

Para utilizar el módulo en un entorno Linux (Fedora), se deben seguir los siguientes pasos:

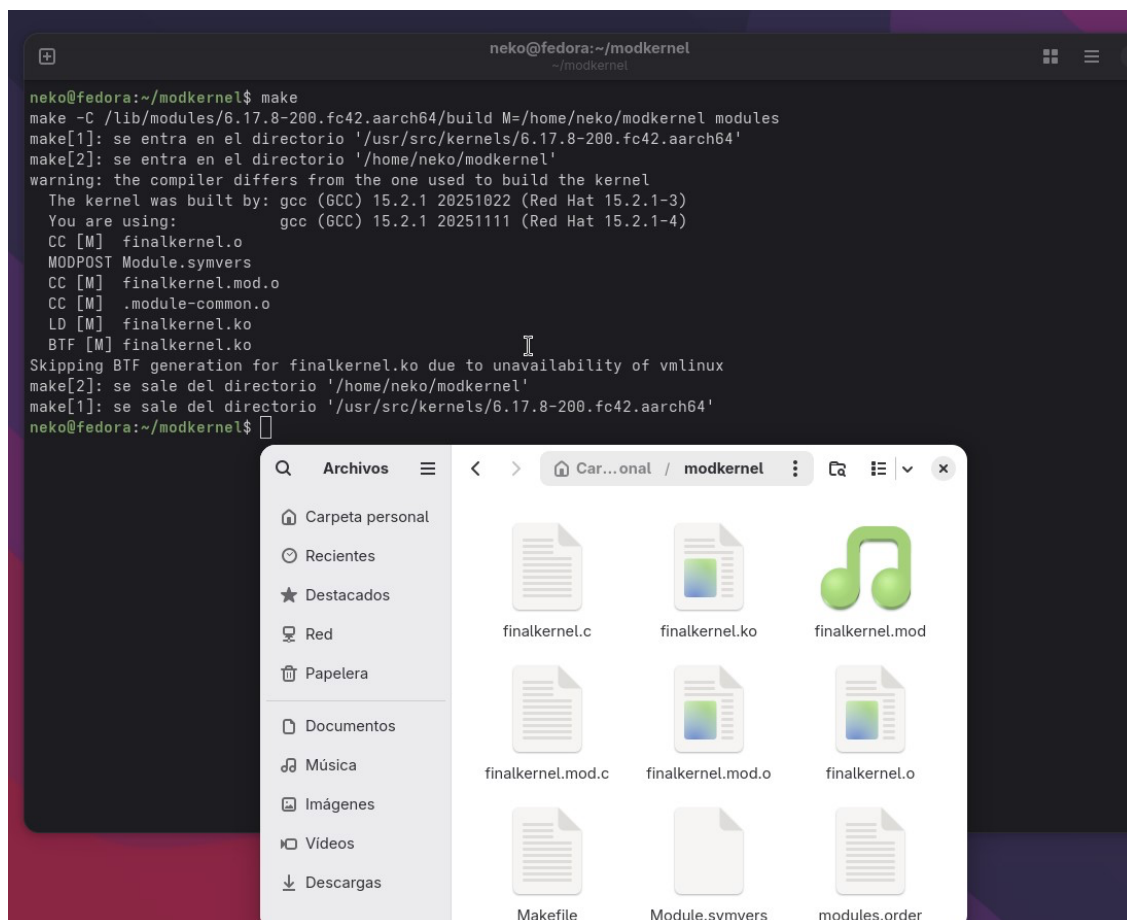
1. **Compilación:** Ejecutar `make` en el directorio del proyecto para generar el fichero objeto `finalkernel.ko`.
2. **Carga:** Insertar el módulo en el kernel con `sudo insmod finalkernel.ko`.
3. **Permisos:** Otorgar permisos de lectura/escritura al dispositivo creado automáticamente: `sudo chmod 666 /dev/finalkernel_dev`.
4. **Pruebas:**
 - Comando `cat /dev/finalkernel_dev` para leer el mensaje.
 - Comando `echo "1" > /dev/finalkernel_dev` para activar el modo flujo.
 - Comando `echo "0" > /dev/finalkernel_dev` para volver al modo normal.
5. **Descarga:** Ejecutar `sudo rmmod finalkernel` para retirar el módulo.

3. Evidencia de Funcionamiento

A continuación se presentan las capturas de pantalla que validan cada etapa del proceso.

Evidencia 1: Compilación Exitosa

Se ejecuta el comando make. El compilador genera los archivos necesarios (.o, .ko, .mod) sin mostrar errores de sintaxis.



Evidencia 2: Carga del Módulo

Se utiliza el comando `sudo insmod finalkernel.ko`. No se muestra salida en terminal, lo cual indica que el comando se ejecutó sin errores inmediatos.

```
neko@fedora:~/modkernel$ sudo insmod finalkernel.ko
neko@fedora:~/modkernel$
```

Evidencia 3: Verificación en Bitácora (dmesg)

Se verifica la carga correcta revisando los logs del kernel con `sudo dmesg | tail`.

- Se observa el mensaje: "FinalKernel: Inicializando con GOTO...", confirmando que se está usando la versión del código con manejo de errores avanzado.
- Se confirma el registro del dispositivo y la asignación del Major Number (505).

```
neko@fedora:~/modkernel$ sudo dmesg | tail
[ 56.153346] usb 1-1: Failed to query (GET_CUR) UVC error code control 2 on unit 1: -32 (exp. 1).
[ 56.705725] input: VMware DnD UInput pointer as /devices/virtual/input/input8
[ 1273.391421] evm: overlay not supported
[ 1514.554841] nvme nvme0: I/O tag 192 (70c0) QID 4 timeout, completion polled
[ 1818.314508] nvme nvme0: I/O tag 36 (0024) QID 4 timeout, completion polled
[ 2487.670929] finalkernel: loading out-of-tree module taints kernel.
[ 2487.670968] finalkernel: module verification failed: signature and/or required key missing - tainting kernel
[ 2487.673872] FinalKernel: Inicializando con GOTO...
[ 2487.673882] FinalKernel: Registrado con Major Number: 505
[ 2487.675594] FinalKernel: Modulo cargado correctamente. Dispositivo: /dev/finalkernel_dev
neko@fedora:~/modkernel$
```


Código anexo del modulo de kernel:

```

/**
 * @file final_ops.c
 * @brief Módulo de Kernel para Examen Final de Sistemas Operativos
 * @author Tu Nombre (Estudiante)
 * @version 1.0
 */

#include <linux/module.h>

#include <linux/fs.h>

#include <linux/kernel.h>

#include <linux/cdev.h>

#include <linux/uaccess.h>

#include <linux/device.h>

#include <linux/kdev_t.h>

#include <linux/slab.h>

#include <linux/err.h> // Necesario para gestionar errores de punteros

// Metadatos del Módulo

MODULE_LICENSE("GPL");

MODULE_DESCRIPTION("Modulo kerneles");

// Constantes

#define DEVICE_NAME "finalkernel_dev" // Nombre del dispositivo en /dev

#define CLASS_NAME "finalkernel_class"

```

```
#define BUFFER_SIZE 1024
```

```
// Variables Globales static int
```

```
dev_major = 0; static struct class
```

```
*exam_class = NULL; static struct cdev
```

```
exam_cdev;
```

```
// Variables de estado static int stream_active = 0; // 0 = Apagado
```

```
(Default), 1 = Encendido static int sent_once = 0; // Control para
```

```
modo normal (no flujo)
```

```
// Buffers static char msg_buffer[BUFFER_SIZE] = "Todo funcionando por
```

```
aquiii!\n"; static char recv_buffer[BUFFER_SIZE];
```

```
// Prototipos de funciones static int dev_open(struct inode *, struct
```

```
file *); static int dev_release(struct inode *, struct file *); static ssize_t
```

```
dev_read(struct file *, char __user *, size_t, loff_t *); static ssize_t
```

```
dev_write(struct file *, const char __user *, size_t, loff_t *);
```

```
// Estructura de operaciones de archivo
```

```
static struct file_operations fops = {
```

```
.open = dev_open,
```

```
.read = dev_read,
```

```
.write = dev_write,
```

```
.release = dev_release,
```

```
};
```

```
/**
```

```
 * @brief Se ejecuta al cargar el módulo (insmod)
```

```
 * AQUI SE APLICA EL CAMBIO PARA USAR GOTO
```

```
 */
```

```
static int __init final_ops_init(void) {    int error = 0; // Variable para
guardar códigos de error    dev_t dev_num; // Variable temporal para
los números de dispositivo
```

```
    printk(KERN_INFO "FinalKernel: Inicializando con GOTO...\n");
```

```
    // 1. Asignar número mayor dinámicamente    error =
```

```
    alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
```

```
    if (error < 0) {
```

```
        printk(KERN_ALERT "Finalkernel: Fallo al asignar numero mayor\n");
```

```
    return error; // Aquí no hay nada que limpiar, retornamos directo
```

```
    }
```

```
    dev_major = MAJOR(dev_num);    printk(KERN_INFO "Finalkernel: Registrado con
Major Number: %d\n", dev_major);
```

```
    // 2. Crear la clase del dispositivo (sysfs)    exam_class =
```

```
    class_create(CLASS_NAME);    if (IS_ERR(exam_class)) {
```

```
        printk(KERN_ALERT "Finalkernel: Fallo al crear la clase\n");
```

```
error = PTR_ERR(exam_class);    goto r_unreg; // <--- SALTO
```

```
A LIMPIEZA NIVEL 1
```

```
}
```

```
// 3. Crear el nodo del dispositivo (/dev/finalkernel_dev)  if
```

```
(device_create(exam_class, NULL, dev_num, NULL, DEVICE_NAME) == NULL) {
```

```
printk(KERN_ALERT "Finalkernel: Fallo al crear el dispositivo\n");
```

```
error = -1;    goto r_class; // <--- SALTO A
```

```
LIMPIEZA NIVEL 2
```

```
}
```

```
// 4. Inicializar cdev y añadirlo al kernel
```

```
cdev_init(&exam_cdev, &fops);  if (cdev_add(&exam_cdev,
```

```
dev_num, 1) == -1) {    printk(KERN_ALERT "Finalkernel:
```

```
Fallo al añadir cdev\n");
```

```
error = -1;
```

```
goto r_device; // <--- SALTO A LIMPIEZA NIVEL 3
```

```
}
```

```
printk(KERN_INFO "Finalkernel: Modulo cargado correctamente. Dispositivo:
/dev/%s\n", DEVICE_NAME);
```

```
return 0; // Éxito total
```

```
// -----
```

```
// ZONA DE LIMPIEZA DE ERRORES (CASCADA)
```

```

// Se ejecutan en orden inverso a la creación
// -----

r_device:

    device_destroy(exam_class, dev_num); // Deshacer paso 3

r_class:

    class_destroy(exam_class);          // Deshacer paso 2

r_unreg:

    unregister_chrdev_region(dev_num, 1); // Deshacer paso 1


return error;
}

/**
 * @brief Se ejecuta al descargar el módulo (rmmod)
 */

static void __exit final_ops_exit(void) {
    dev_t dev_num = MKDEV(dev_major, 0);
    cdev_del(&exam_cdev);
    device_destroy(exam_class, dev_num);
    class_destroy(exam_class);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "Finalkernel: Modulo descargado. Hasta luego!\n");
}

```

```

/**
 * @brief Se ejecuta cuando se abre el archivo (cat /dev/final_ops_dev)
 */

static int dev_open(struct inode *inodep, struct file *filep) {
    printk(KERN_INFO "Finalkernel: Dispositivo abierto\n");

    // Reiniciamos la variable de 'ya enviado' cada vez que se abre el archivo

    // a menos que estemos en modo stream

    if (!stream_active) {        sent_once = 0;

        }

        return 0;
    }

}

/**
 * @brief Lectura: Envía datos del Kernel al Usuario
 */

static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t *offset) {
    int errors = 0;    int msg_len = strlen(msg_buffer);

    // Lógica de Modos:

    if (!stream_active && sent_once) {

return 0; // EOF, detiene el cat

    }

    errors = copy_to_user(buffer, msg_buffer, msg_len);

```

```

    if (errors == 0) {        if (!stream_active) {        sent_once = 1; // Marcamos como
enviado para que la proxima lectura de 0        printk(KERN_INFO "Finalkernel:
Mensaje enviado a usuario (Modo Normal)\n");

        } else {

            printk(KERN_INFO "Finalkernel: Mensaje enviado a usuario (Modo Flujo)\n");

        }

        return msg_len;

    } else {

        printk(KERN_ALERT "Finalkernel: Error enviando datos al usuario\n");
return -EFAULT;

    }

}

/**
 * @brief Escritura: Recibe datos del Usuario (echo "1" > /dev/...)
 */

static ssize_t dev_write(struct file *filep, const char __user *buffer, size_t len, loff_t *offset)
{
    if (len > BUFFER_SIZE - 1) len = BUFFER_SIZE - 1;

    if (copy_from_user(recv_buffer, buffer, len) != 0) {
return -EFAULT;

    }

    recv_buffer[len] = '\0'; // Asegurar terminación de cadena

```

```

// Imprimir en bitácora lo recibido   printk(KERN_INFO "FINAL_OPS:
Recibido desde usuario: %s", recv_buffer);

// Lógica de control de flujo
if (recv_buffer[0] == '1') {
    stream_active = 1;

    printk(KERN_INFO "Finalkernel: ==> MODO FLUJO ACTIVADO (CTRL-C para detener) <==\n");
} else if (recv_buffer[0] == '0') {
    stream_active = 0;    sent_once
= 0;    printk(KERN_INFO
"Finalkernel: ==> MODO FLUJO
DESACTIVADO (Default) <==\n");

} else {

    printk(KERN_INFO "Finalkernel: Comando desconocido. Use '1' para flujo, '0' para
normal.\n");

}

return len;
}

static int dev_release(struct inode *inodep, struct file *filep) {
    printk(KERN_INFO "Finalkernel: Dispositivo cerrado\n");

    return 0;
}

module_init(final_ops_init); module_exit(final_ops_exit);

```